Ejercicios POO Java JS

Necesito generar ejercicios para comprender conceptos de programación orientada a objetos con JAVA y JS

Perfecto, te armo una **batería de ejercicios progresivos (tipo TP)** para que tus alumnos comprendan bien **POO (Programación Orientada a Objetos)** tanto en **Java** como en **JavaScript**.
Lo voy a estructurar **como docente**: simple → intermedio → aplicado.

🧠 TP: Introducción a POO (Java + JavaScript)

🎯 Objetivos

- Comprender **clases y objetos**
- Usar **atributos y métodos**
- Aplicar **encapsulamiento**
- Introducir **herencia**
- Comparar implementación en Java vs JS

🟢 PARTE 1: CLASES Y OBJETOS

🧩 Ejercicio 1: Clase Persona

📌 Consigna

Crear una clase `Persona` con:

- nombre
- edad

Método:

- saludar()

☕ Java

```Java
public class Persona {
    String nombre;
    int edad;
```

```
    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

Java

```java
public class Main {
    public static void main(String[] args) {
        Persona p = new Persona();
        p.nombre = "Juan";
        p.edad = 25;

        p.saludar();
    }
}
```

🌐 JavaScript

JavaScript

```javascript
class Persona {
    constructor(nombre, edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    saludar() {
        console.log("Hola, soy " + this.nombre);
    }
}

let p = new Persona("Juan", 25);
p.saludar();
```

🟡 PARTE 2: ENCAPSULAMIENTO

🧩 Ejercicio 2: Cuenta Bancaria

📌 Consigna

Crear una clase `Cuenta` con:

- saldo (privado)
- métodos: depositar(), retirar(), mostrarSaldo()

☕ Java

```java
public class Cuenta {
    private double saldo;

    public void depositar(double monto) {
        saldo += monto;
    }

    public void retirar(double monto) {
        if (monto <= saldo) {
            saldo -= monto;
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

🌐 JavaScript

```javascript
class Cuenta {
    #saldo = 0;

    depositar(monto) {
        this.#saldo += monto;
    }

    retirar(monto) {
        if (monto <= this.#saldo) {
            this.#saldo -= monto;
        }
    }

    mostrarSaldo() {
        console.log(this.#saldo);
    }
}
```

🟠 PARTE 3: CONSTRUCTORES

🧩 Ejercicio 3: Producto

📌 Consigna

Crear clase `Producto` con:

- nombre
- precio

Inicializar con constructor
Método: mostrarInfo()

👉 (Lo dejás como ejercicio sin resolver o lo resolvemos después para docente)

🔵 PARTE 4: HERENCIA

🧩 Ejercicio 4: Vehículos

📌 Consigna

Clase base:

- Vehiculo → marca, modelo

Clase hija:

- Auto → puertas

Método:

- mostrarInfo()

☕ Java

```java
class Vehiculo {
    String marca;
    String modelo;
}

class Auto extends Vehiculo {
    int puertas;
}
```

🌐 JavaScript

JavaScript

```javascript
class Vehiculo {
    constructor(marca, modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }
}

class Auto extends Vehiculo {
    constructor(marca, modelo, puertas) {
        super(marca, modelo);
        this.puertas = puertas;
    }
}
```

🔴 PARTE 5: EJERCICIO INTEGRADOR

🧩 Ejercicio 5: Sistema simple de alumnos

📌 Consigna

Crear:

- Clase `Alumno`
  - nombre
  - notas (array)
- Método:
  - agregarNota()
  - calcularPromedio()

💡 Extensión:

- Mostrar si aprueba (>= 6)

🧪 PARTE 6: EJERCICIOS PARA PRACTICAR (SIN RESOLVER)

Nivel básico

1. Clase `Mascota` → nombre, especie → método mostrar()
2. Clase `Libro` → título, autor → método describir()

Nivel intermedio

3. Clase `Rectangulo` → base, altura → calcular área
4. Clase `Empleado` → nombre, sueldo → aumentar sueldo %

Nivel avanzado

5. Clase `Usuario` con contraseña privada
6. Sistema de carrito de compras (array de productos)

👩‍🏫 IDEAS DOCENTES (MUY ÚTIL PARA VOS)

Podés hacer:

- Comparación directa:
  - Java → tipado fuerte
  - JS → dinámico
- Mostrar:
  - `this` en ambos lenguajes
  - diferencias de encapsulación

🚀 Si querés lo llevamos a otro nivel

Puedo armarte:
✅ TP completo en PDF para alumnos
✅ PPT tipo clase
✅ Ejercicios con **resolución paso a paso docente**
✅ Versión con **Spring Boot (tu proyecto banco)**
✅ Integrarlo con tu **mapa Leaflet o sistema web**

Decime cómo lo querés ahora 👇
👉 "modo clase", "modo examen" o "modo práctica guiada"

como uso Java sin tipado y asignado tipo después para no cometer errores con scanner y método next?

Buena pregunta — y te corrijo algo importante de entrada:

👉 En **Java NO existe "usar sin tipado"** como en JavaScript.
Java es un lenguaje de **tipado estático**, siempre tenés que declarar tipos.

Pero 👇
sí hay formas de trabajar **más flexible y evitar errores con** `Scanner` **y** `next()`.

🧠 EL PROBLEMA REAL (lo que te está pasando)

Seguramente te pasa esto:

```java
Scanner sc = new Scanner(System.in);

int edad = sc.nextInt();   // ❌ rompe si el usuario escribe texto
String nombre = sc.next(); // ❌ corta en espacios
```

👉 Problemas típicos:

- `InputMismatchException`
- mezcla de `nextInt()` + `nextLine()`
- cadenas incompletas

✅ SOLUCIÓN PROFESIONAL (la que se usa SIEMPRE)

🔹 1. Leer TODO como String

👉 Este es el truco clave:

```java
Scanner sc = new Scanner(System.in);

String entrada = sc.nextLine();
```

💡 Esto:

- nunca rompe
- acepta espacios
- es seguro

🔹 2. Convertir después (parseo)

```java
```

```
int edad = Integer.parseInt(entrada);
```

🟢 EJEMPLO CORRECTO COMPLETO

Java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese su edad: ");
        String input = sc.nextLine();

        int edad = Integer.parseInt(input);

        System.out.println("Edad: " + edad);
    }
}
```

🛑 CONTROL DE ERRORES (clave para alumnos)

Java

```java
try {
    int edad = Integer.parseInt(input);
    System.out.println("Edad: " + edad);
} catch (NumberFormatException e) {
    System.out.println("Error: ingresá un número válido");
}
```

🔵 ¿Esto es “tipado dinámico”?

👉 No exactamente.

Pero se parece a cómo funciona en JavaScript:

JavaScript

```javascript
let edad = prompt("Edad"); // string
edad = Number(edad);       // conversión después
```

💡 En Java estás haciendo lo mismo manualmente.

🔥 REGLA DE ORO PARA CLASES

💬 Decís a tus alumnos:

👉 *"En Java SIEMPRE lean con nextLine() y después conviertan"*

⚠️ LO QUE NO HAY QUE HACER

```java
int edad = sc.nextInt();   // ❌ peligroso en práctica
double nota = sc.nextDouble();
```

👉 porque:

- rompe con entradas incorrectas
- deja basura en el buffer
- genera errores difíciles de entender

🧩 EXTRA: limpiar buffer (por si lo usan igual)

```java
int edad = sc.nextInt();
sc.nextLine(); // limpia el salto de línea
```

🧪 PATRÓN REUTILIZABLE (MUY DOCENTE)

```java
public static int leerEntero(Scanner sc) {
    while (true) {
        try {
            return Integer.parseInt(sc.nextLine());
        } catch (Exception e) {
            System.out.print("Número inválido, reintente: ");
        }
    }
}
```

Uso:

```java
```

```
int edad = leerEntero(sc);
```

🚀 CONCLUSIÓN CLARA

👉 No podés evitar el tipado en Java
👉 Pero podés trabajar como si fuera "dinámico" leyendo Strings
👉 Y eso elimina el 90% de errores con Scanner

Si querés, te armo 👇
✅ una **clase completa de entrada de datos para alumnos**
✅ ejercicios tipo examen con Scanner
✅ o lo integramos a tu proyecto del banco (login + input seguro)

1/2

pero se puede usar tipos parametrizados y/o un genérico que después yo decida el tipo

Muy buena pregunta — acá ya estás entrando en un nivel más avanzado 👏

👉 **Respuesta corta:**
Sí, podés usar **genéricos** (`<T>`), pero **NO te permiten decidir el tipo "después" en tiempo de ejecución** como en JavaScript.
Los genéricos en Java son **tipados en tiempo de compilación**.

🧠 IDEA CLAVE

👉 En Java:

- ✔️ `T` se define en compilación
- ❌ no podés cambiar el tipo dinámicamente en runtime
- ✔️ sirven para reutilizar código de forma segura

🔵 EJEMPLO SIMPLE DE GENÉRICO

Java

```java
public class Caja<T> {
    private T valor;

    public void set(T valor) {
```

```
        this.valor = valor;
    }

    public T get() {
        return valor;
    }
}
```

Uso:

```Java
Caja<Integer> c1 = new Caja<>();
c1.set(10);

Caja<String> c2 = new Caja<>();
c2.set("Hola");
```

👉 Acá el tipo ya quedó definido.

🟡 LO QUE VOS QUERÉS HACER (entrada dinámica)

Vos querés algo tipo:

👉 "leo y después decido si es int, double, String..."

💡 Eso **NO lo hacen los genéricos**
👉 lo hacés con lógica + parseo

🟢 SOLUCIÓN REAL: método genérico con conversión

Podés combinar genéricos + función:

```Java
import java.util.Scanner;
import java.util.function.Function;

public class Input {

    public static <T> T leer(Scanner sc, Function<String, T> parser) {
        while (true) {
            try {
                return parser.apply(sc.nextLine());
            } catch (Exception e) {
                System.out.print("Entrada inválida, reintente: ");
            }
        }
```

```
    }
}
```

🔥 USO (MUY POTENTE)

```java
Scanner sc = new Scanner(System.in);

int edad = Input.leer(sc, Integer::parseInt);
double altura = Input.leer(sc, Double::parseDouble);
String nombre = Input.leer(sc, s -> s);
```

💡 ESTO ES LO MÁS CERCANO A "DINÁMICO" EN JAVA

👉 Vos decidís el tipo en el momento de llamar
👉 pero el compilador sigue controlando todo

🔴 OPCIÓN MÁS FLEXIBLE (pero menos segura)

Usar `Object`:

```java
Object valor = sc.nextLine();
```

Después:

```java
if (valor instanceof String) {
    ...
}
```

❌ Problema:

- perdés tipado fuerte
- más propenso a errores

🟣 OPCIÓN INTERMEDIA: usar `var` (Java 10+)

```java
```

```
var entrada = sc.nextLine(); // sigue siendo String
```

👉 Esto **no es dinámico**, solo infiere tipo

🧪 BONUS DOCENTE (explicación clara para alumnos)

Podés decir:

👉 "Java no es dinámico como JS, pero podemos simularlo leyendo String y transformando después"

🚀 CONCLUSIÓN

✔️ Genéricos sirven para reutilizar código
❌ No sirven para decidir tipo en runtime
✔️ La mejor práctica:
👉 `nextLine()` + parseo + función genérica

Si querés, te armo 👇
✅ una **clase Input completa tipo librería para tus TPs**
✅ ejercicios donde los alumnos construyen esto
✅ o una versión integrada con tu sistema bancario (validación de datos real)

eso quería, declarando como var y luego asignar el tipo

Perfecto, ahora te explico bien porque acá hay una **confusión muy común (y clave para enseñar bien Java)** 👇

🧠 ❗ IMPORTANTE: `var` NO es tipado dinámico

👉 En Java:

```java
var x = "Hola";
```

🔒 **x YA ES String para siempre**

❌ No podés hacer:

```java
x = 10; // ERROR
```

👉 Porque Java es **tipado estático con inferencia**, no dinámico.

🔵 ¿Entonces qué hace `var`?

👉 Solo evita escribir el tipo explícito:

```java
String nombre = "Juan";
```

es lo mismo que:

```java
var nombre = "Juan"; // el compilador decide: String
```

🟡 LO QUE VOS QUERÉS HACER (tipo "JS")

En JavaScript:

```javascript
let x = "Hola";
x = 10; // OK
```

👉 En Java esto **NO existe**.

🟢 LO MÁS PARECIDO QUE PODÉS HACER

✔️ Opción 1: usar `var` + parseo

```java
var entrada = sc.nextLine(); // String

int edad = Integer.parseInt(entrada);
```

👉 Esto es lo correcto y seguro.

✔️ Opción 2: usar `Object` (dinámico pero peligroso)

```java
Object valor = sc.nextLine();

// después decidís
valor = Integer.parseInt((String) valor);
```

👉 Problemas:

- casts
- errores en runtime
- poco recomendable para alumnos

✔️ Opción 3: tu mejor enfoque (el que buscabas)

👉 Usar `var` + método genérico 👇

```java
var edad = Input.leer(sc, Integer::parseInt);
var nombre = Input.leer(sc, s -> s);
```

💡 Acá:

- usás `var`
- el tipo se infiere automáticamente
- mantenés seguridad

🔥 EJEMPLO COMPLETO (ESTILO DOCENTE)

```java
import java.util.Scanner;
import java.util.function.Function;

public class Main {

    public static <T> T leer(Scanner sc, Function<String, T> parser) {
        while (true) {
            try {
                return parser.apply(sc.nextLine());
            } catch (Exception e) {
                System.out.print("Error, reintente: ");
            }
        }
    }

    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
```

🧩 IDEA CLAVE PARA ENSEÑAR

💬 Podés decir:

👉 "Java NO es dinámico, pero podemos hacerlo flexible usando `var` + funciones"

🚀 RESUMEN FINAL

✔ `var` NO permite cambiar tipo
✔ el tipo se define al momento de asignar
✔ la mejor práctica:
👉 `nextLine()` + parseo + genéricos + `var`

Si querés, te armo algo muy potente para tus clases 👇
✅ comparación directa Java vs JS (ejercicios)
✅ TP donde los alumnos construyen su propio Input helper
✅ errores típicos con Scanner (tipo examen con trampas 😄)

> Hagamos un resumen, una primera parte con lectura de datos por consola y conversión a distintos tipos y una segunda con ejemplos de aplicación de POO en JAVA

Perfecto, te armo un **resumen claro tipo clase/TP** dividido en dos partes, ideal para tus alumnos 👇

🧠 PARTE 1: Entrada de datos por consola (Java)

🎯 Objetivo

- Leer datos correctamente
- Evitar errores con `Scanner`
- Convertir a distintos tipos

❌ Problema típico

```java
Scanner sc = new Scanner(System.in);

int edad = sc.nextInt(); // ❌ puede fallar
String nombre = sc.next(); // ❌ corta en espacios
```

👉 Problemas:

- errores si ingresan texto
- conflictos con saltos de línea

✅ SOLUCIÓN RECOMENDADA

👉 Leer siempre como `String`:

```java
Scanner sc = new Scanner(System.in);

String entrada = sc.nextLine();
```

🔄 Conversión de tipos

```java
int numero = Integer.parseInt(entrada);
double decimal = Double.parseDouble(entrada);
```

🟢 Ejemplo completo

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese nombre: ");
        String nombre = sc.nextLine();

        System.out.print("Ingrese edad: ");
        int edad = Integer.parseInt(sc.nextLine());

        System.out.println("Hola " + nombre + ", tenés " + edad + " años");
```

```
    }
}
```

🛡️ Manejo de errores

```Java
try {
    int edad = Integer.parseInt(sc.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Error: ingresar número válido");
}
```

🔥 Método reutilizable (nivel docente)

```Java
import java.util.function.Function;

public static <T> T leer(Scanner sc, Function<String, T> parser) {
    while (true) {
        try {
            return parser.apply(sc.nextLine());
        } catch (Exception e) {
            System.out.print("Dato inválido, reintente: ");
        }
    }
}
```

Uso:

```Java
var edad = leer(sc, Integer::parseInt);
var sueldo = leer(sc, Double::parseDouble);
var nombre = leer(sc, s -> s);
```

💡 Idea clave

👉 Java NO es dinámico

👉 pero podés simular flexibilidad con:

- `nextLine()`
- parseo
- genéricos

- `var`

🧱 PARTE 2: Programación Orientada a Objetos (POO)

🎯 Objetivo

- Comprender clases, objetos y métodos
- Aplicar encapsulamiento y herencia

🟢 1. Clase y objeto

Java

```java
public class Persona {
    String nombre;
    int edad;

    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

Uso:

Java

```java
Persona p = new Persona();
p.nombre = "Ana";
p.edad = 30;
p.saludar();
```

🟡 2. Encapsulamiento

Java

```java
public class Cuenta {
    private double saldo;

    public void depositar(double monto) {
        saldo += monto;
    }

    public double getSaldo() {
        return saldo;
    }
}
```

👉 Protege los datos

🔵 3. Constructor

Java

```java
public class Producto {
    String nombre;
    double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public void mostrar() {
        System.out.println(nombre + " - $" + precio);
    }
}
```

🟣 4. Herencia

Java

```java
class Vehiculo {
    String marca;
    String modelo;
}

class Auto extends Vehiculo {
    int puertas;
}
```

🔴 5. Ejemplo integrador

Java

```java
import java.util.ArrayList;

public class Alumno {
    String nombre;
    ArrayList<Integer> notas = new ArrayList<>();

    void agregarNota(int nota) {
        notas.add(nota);
    }
```

```
    double promedio() {
        int suma = 0;
        for (int n : notas) {
            suma += n;
        }
        return (double) suma / notas.size();
    }
```

🧩 ACTIVIDAD PROPUESTA

📝 Ejercicio integrador

Crear un programa que:

1. Pida por consola:
   - nombre del alumno
   - cantidad de notas
2. Cargue las notas usando el método `leer()`
3. Muestre:
   - promedio
   - si aprueba (>= 6)

🚀 CIERRE (IDEA CLAVE PARA CLASE)

👉 Entrada segura:

> "Siempre leer como String y convertir"

👉 POO:

> "Modelamos el mundo real con clases y objetos"

💬 Si querés lo siguiente

Puedo armarte 👇

✅ PPT lista para clase
✅ PDF tipo guía para alumnos
✅ TP evaluativo con consigna + solución docente
✅ versión con tu sistema bancario (mucho más realista)

Supongamos que leo con next el numero 8,4a y lo quiero almacenar como double, entonces hago un ciclo hasta que la entrada no sea válida verdad?

Exacto 👌 — vas por el camino correcto, pero te ajusto la idea para que quede **bien sólida (nivel docente + práctica real)**.

🧠 ✔️ IDEA CLAVE

👉 Si el usuario ingresa algo como:

```
8,4a
```

❌ **NO es un `double` válido**

Entonces:

👉 **sí**, tenés que usar un **ciclo hasta que la entrada sea válida**

⚠️ PERO... no uses `next()` para esto

👉 Usá siempre:

```java
sc.nextLine()
```

Porque:

- `next()` corta en espacios
- complica validaciones
- es menos predecible

🟢 SOLUCIÓN CORRECTA (patrón estándar)

```java
Scanner sc = new Scanner(System.in);
double numero;

while (true) {
    System.out.print("Ingrese un número decimal: ");
    String input = sc.nextLine();

    try {
```

```
            numero = Double.parseDouble(input);
            break; // ✓ válido → salgo del ciclo
        } catch (NumberFormatException e) {
            System.out.println("Error: número inválido");
        }
    }

    System.out.println("Número ingresado: " + numero);
```

🔍 ¿Qué pasa con `"8,4a"`?

👉 `Double.parseDouble("8,4a")`
→ lanza `NumberFormatException`

✔️ entonces:

- entra al `catch`
- vuelve a pedir el dato

🟡 DETALLE IMPORTANTE (muy realista en Argentina AR)

👉 El usuario puede escribir:

```
8,4
```

❌ Eso también falla en Java
(Java espera punto `.`)

✔️ Solución práctica

‹/› Java

```
input = input.replace(",", ".");
```

🔵 Versión mejorada

‹/› Java

```
while (true) {
    System.out.print("Ingrese un número decimal: ");
    String input = sc.nextLine().replace(",", ".");
```

```
    try {
        double numero = Double.parseDouble(input);
        System.out.println("OK: " + numero);
        break;
    } catch (NumberFormatException e) {
        System.out.println("Error: número inválido");
    }
}
```

🧪 OPCIÓN MÁS AVANZADA (validar antes)

```Java
if (input.matches("-?\\d+(\\.\\d+)?")) {
    double numero = Double.parseDouble(input);
}
```

👉 pero para alumnos:
❌ suele ser más complejo
✔️ mejor usar try-catch

🔥 PATRÓN FINAL PARA ENSEÑAR

💬 Podés decir:

👉 "Pedimos el dato → intentamos convertir → si falla, repetimos"

🚀 CONCLUSIÓN

✔️ Sí, usás un `while` hasta que sea válido
✔️ Siempre leer como `String`
✔️ Convertir con `parseDouble`
✔️ Manejar error con `try-catch`

Si querés, te armo 👇
✅ una **clase Input completa (enteros, double, texto)**
✅ ejercicios con errores típicos (para examen 👀)
✅ versión integrada a tu sistema bancario (depósitos/transferencias con validación real)

yo quería que use un if no numeric o algo asi, sin try catch y sin expresiones regulares

Perfecto 👌 ahora entendí exactamente lo que buscás:

👉 **validar "a mano" sin `try-catch` y sin regex**
👉 algo tipo "si no es numérico, repetir"

Se puede hacer ✔ pero con una **función que verifique carácter por carácter**.

🧠 IDEA

👉 En vez de intentar convertir, primero validás:

- dígitos ✔
- un solo `.` o `,` ✔
- opcional `-` al inicio ✔

🟢 FUNCIÓN SIN regex ni try-catch

Java

```java
public static boolean esDoubleValido(String input) {

    if (input == null || input.isEmpty()) return false;

    int puntos = 0;

    for (int i = 0; i < input.length(); i++) {
        char c = input.charAt(i);

        if (c == '-' && i == 0) continue;

        if (c == '.' || c == ',') {
            puntos++;
            if (puntos > 1) return false;
            continue;
        }

        if (!Character.isDigit(c)) {
```

🔵 USO CON `if` + `while`

Java

```java
Scanner sc = new Scanner(System.in);
String input;

while (true) {
    System.out.print("Ingrese un número: ");
    input = sc.nextLine();

    if (esDoubleValido(input)) {
        break;
    } else {
        System.out.println("Error: no es un número válido");
    }
}

// recién ahora convierto
double numero = Double.parseDouble(input.replace(",", "."));

System.out.println("Número: " + numero);
```

🔍 ¿Qué valida esto?

✔ `123`
✔ `-10`
✔ `3.14`
✔ `8,4`

❌ `8,4a`
❌ `abc`
❌ `3..5`

🧩 EXPLICACIÓN DOCENTE SIMPLE

💬 Podés decir:

👉 “Recorremos el texto carácter por carácter y verificamos si cumple reglas de número”

🔥 VENTAJAS DE ESTE ENFOQUE

✔ No usa excepciones
✔ No usa regex
✔ Es didáctico (se entiende cómo funciona)
✔ Ideal para primeros niveles

⚠️ LIMITACIONES

❌ No cubre casos complejos (ej: `1e5`)
❌ No valida formato internacional avanzado

👉 pero para clase básica → PERFECTO

🚀 BONUS (versión más simple todavía)

Si querés algo más básico (solo enteros):

```Java
public static boolean esEntero(String input) {
    for (char c : input.toCharArray()) {
        if (!Character.isDigit(c)) return false;
    }
    return true;
}
```

✅ CONCLUSIÓN

✔️ Sí, podés usar `if` sin try-catch
✔️ Validás primero → convertís después
✔️ Es una excelente práctica para enseñar lógica

Si querés, el siguiente paso ideal para tu clase 👇
✅ hacer que los alumnos construyan esta función
✅ darles inputs inválidos para probar
✅ integrar esto en el TP de POO (ej: carga de cuentas, alumnos, etc.)

me parece más simple con try catch

Totalmente 👍 — y te diría que **es la mejor decisión en Java real**.

🧠 ✔️ Conclusión clara

👉 `try-catch` **es más simple, más seguro y más profesional**

Porque:

- Java ya valida por vos
- evitás errores de lógica
- el código es más corto y claro

🟢 Patrón recomendado (EL que deberías enseñar)

```java
Scanner sc = new Scanner(System.in);
double numero;

while (true) {
    System.out.print("Ingrese un número: ");
    String input = sc.nextLine().replace(",", ".");

    try {
        numero = Double.parseDouble(input);
        break; // ✓ válido
    } catch (NumberFormatException e) {
        System.out.println("Error: número inválido");
    }
}

System.out.println("Número: " + numero);
```

🔥 Por qué es mejor que validar a mano

Método	Complejidad	Robustez	Recomendado
if + validación	Alta	Media	❌
regex	Media	Alta	⚠️
try-catch	Baja	Muy alta	✅

🧩 IDEA DOCENTE CLAVE

💬 Podés explicarlo así:

👉 *"En Java no adivinamos si es válido… lo intentamos y, si falla, lo manejamos"*

⚠️ Algo importante para enseñar bien

Muchos alumnos piensan:

👉 "las excepciones son malas"

❌ Error

👉 En este caso son **la forma correcta de validar entrada**

🚀 BONUS: versión reutilizable (para tus TPs)

Java
```java
public static double leerDouble(Scanner sc) {
    while (true) {
        try {
            return Double.parseDouble(sc.nextLine().replace(",", "."));
        } catch (Exception e) {
            System.out.print("Número inválido, reintente: ");
        }
    }
}
```

Uso:

Java
```java
double monto = leerDouble(sc);
```

🎯 CIERRE

✔️ Sí, elegiste el mejor enfoque
✔️ Es más limpio
✔️ Es más cercano a código profesional
✔️ Es ideal para enseñar

Si querés, en el siguiente paso te armo 👇
✅ un TP donde mezclen **input + POO (cuentas, alumnos, etc.)**
✅ con validación real usando este patrón
✅ y errores típicos tipo examen 😄

dale

Dale. Te dejo un **TP/guía** en dos partes, como veníamos armando: primero **lectura segura por consola** y después **aplicación de POO en Java**.

Guía práctica: entrada de datos + POO en Java

Parte 1: Lectura de datos por consola y conversión

Idea principal

En Java conviene **leer todo como texto** con `nextLine()` y después **convertir** al tipo necesario.

Eso evita problemas clásicos de:

- `nextInt()` mezclado con `nextLine()`
- entradas inválidas
- cortes de texto con espacios

1.1. Ejemplo básico de lectura

Java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese su nombre: ");
        String nombre = sc.nextLine();

        System.out.println("Hola " + nombre);
    }
}
```

1.2. Conversión a entero

Java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Ingrese su edad: ");
        int edad = Integer.parseInt(sc.nextLine());

        System.out.println("Edad: " + edad);
    }
}
```

1.3. Conversión a double con validación

Java

```
import java.util.Scanner;

public class Main {

    public static double leerDouble(Scanner sc) {
        while (true) {
            System.out.print("Ingrese un número decimal: ");
            String input = sc.nextLine().replace(",", ".");

            try {
                return Double.parseDouble(input);
            } catch (NumberFormatException e) {
                System.out.println("Error: número inválido. Reintente.");
            }
        }
    }

    public static void main(String[] args) {
```

1.4. Conversión a int con validación

Java

```
import java.util.Scanner;

public class Main {

    public static int leerEntero(Scanner sc) {
        while (true) {
            System.out.print("Ingrese un número entero: ");
            String input = sc.nextLine();

            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.println("Error: entero inválido. Reintente.");
            }
```

```
        }
    }

    public static void main(String[] args) {
```

1.5. Método reutilizable para texto

```Java
import java.util.Scanner;

public class Main {

    public static String leerTexto(Scanner sc, String mensaje) {
        System.out.print(mensaje);
        return sc.nextLine();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String ciudad = leerTexto(sc, "Ingrese su ciudad: ");
        System.out.println("Ciudad: " + ciudad);
    }
}
```

1.6. Resumen de esta parte

Reglas prácticas

- Para texto: `nextLine()`
- Para enteros: `Integer.parseInt(nextLine())`
- Para decimales: `Double.parseDouble(nextLine())`
- Para entradas inválidas: `try-catch` dentro de un `while`

Idea docente

> En Java, primero se lee el dato como texto y luego se convierte al tipo necesario.

Parte 2: Aplicación de POO en Java

Conceptos a trabajar

- clase

- objeto
- atributo
- método
- constructor
- encapsulamiento

2.1. Clase y objeto: ejemplo simple

Clase Persona

Java
```java
public class Persona {
    String nombre;
    int edad;

    void saludar() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.")
    }
}
```

Uso en `Main`

Java
```java
public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona();

        p1.nombre = "Lucía";
        p1.edad = 22;

        p1.saludar();
    }
}
```

2.2. Uso de constructor

Java
```java
public class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
```

```
        this.edad = edad;
    }

    void saludar() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.")
    }
}
```

Java

```
public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona("Martín", 30);
        p1.saludar();
    }
}
```

2.3. Encapsulamiento

Acá protegemos los atributos con `private`.

Java

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public void depositar(double monto) {
        saldo += monto;
    }

    public void retirar(double monto) {
        if (monto <= saldo) {
            saldo -= monto;
        } else {
            System.out.println("Saldo insuficiente.");
```

Uso

Java

```
public class Main {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Ana", 1000);
```

```
        cuenta.depositar(500);
        cuenta.retirar(300);
        cuenta.mostrarSaldo();
    }
}
```

2.4. Ejemplo integrador con entrada por consola + POO

Clase Alumno

Java

```java
public class Alumno {
    private String nombre;
    private double nota1;
    private double nota2;

    public Alumno(String nombre, double nota1, double nota2) {
        this.nombre = nombre;
        this.nota1 = nota1;
        this.nota2 = nota2;
    }

    public double calcularPromedio() {
        return (nota1 + nota2) / 2;
    }

    public void mostrarResultado() {
        double promedio = calcularPromedio();
        System.out.println("Alumno: " + nombre);
```

Main con lectura segura

Java

```java
import java.util.Scanner;

public class Main {

    public static double leerDouble(Scanner sc, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            String input = sc.nextLine().replace(",", ".");

            try {
                return Double.parseDouble(input);
            } catch (NumberFormatException e) {
                System.out.println("Error: número inválido. Reintente.");
            }
        }
    }
```

```
    public static void main(String[] args) {
```

Actividades propuestas para alumnos

Ejercicio 1

Hacer un programa que pida:

- nombre
- edad
- altura

Luego mostrar todos los datos.

Ejercicio 2

Crear una clase `Mascota` con:

- nombre
- especie
- edad

Método:

- `mostrarDatos()`

Ejercicio 3

Crear una clase `Producto` con:

- nombre
- precio

Métodos:

- `mostrarInfo()`
- `aumentarPrecio(double porcentaje)`

Ejercicio 4

Crear una clase `Empleado` con:

- nombre
- sueldo

Método:

- `aumentarSueldo(double monto)`

Ejercicio 5

Crear una clase `CuentaBancaria` y desde `Main`:

- pedir titular
- pedir saldo inicial
- pedir monto a depositar
- mostrar saldo final

Cierre conceptual

Entrada de datos

En Java, lo más seguro es:

1. leer con `nextLine()`
2. convertir con `parseInt()` o `parseDouble()`
3. validar con `try-catch`

POO

La POO permite representar entidades del mundo real usando:

- atributos para guardar datos
- métodos para definir comportamiento
- objetos como instancias de una clase

Mini resumen para tus alumnos

```
- Scanner se usa para leer datos.
- nextLine() es la forma más segura de entrada.
- Integer.parseInt() convierte a entero.
- Double.parseDouble() convierte a decimal.
- Una clase define atributos y métodos.
- Un objeto es una instancia de una clase.
- El constructor inicializa objetos.
- private protege los atributos.
```

Si querés, en el próximo mensaje te lo convierto en **formato de apunte docente prolijo**, o directamente en **TP con consignas y soluciones**.